

Glossa Lab: A Computational Toolkit for Ancient Script Analysis and Decipherment

Project Report and Collaboration Proposal

BitConcepts — 2026-04-01

1. Project Overview

Glossa Lab is a proprietary, cross-platform research toolkit for computational analysis and decipherment of ancient scripts, developed by BitConcepts. Built in Python with a FastAPI backend, it combines statistical analysis, structural pattern detection, and automated decipherment in a single integrated platform. The toolkit incorporates patented technologies and is not publicly available at this time.

The toolkit was designed with the Indus script as its primary target, but has been validated on synthetic ciphers and the historically deciphered Ugaritic script to establish baseline performance before attempting the undeciphered Indus corpus.

2. Analysis Pipelines (11 total)

Pipeline	Function	Status
block_entropy	Rao et al. (2009) normalised block entropy H_N	Implemented
char_freq	Zipf distribution, rank-frequency, Zipf exponent	Implemented
kandles	Phonetic color-coding and grids (Merkur patent)	Implemented
positional	Position-specific sign frequencies (initial/medial/terminal)	Implemented
sign_cluster	Distributional clustering (Kober method)	Implemented
paradigm	Inflectional paradigm detection (Ventris/Kober)	Implemented
cooccurrence	Sign co-occurrence network + community detection	Implemented
numerals	Numeral sign identification from distributional behavior	Implemented
decipher	Substitution cipher cracking (hill climbing + trigrams)	Implemented
hypothesis	Iterative hypothesis-driven decipherment engine	Implemented
CPSC projection	Multi-constraint decipherment (optional module)	Implemented

Table 1. Analysis pipelines.

3. Validated Results

3.1 Block Entropy Replication (Rao et al. 2009)

We replicated the block entropy analysis across 9 corpora (English, Tamil, Sanskrit, Indus, DNA, Fortran, random, ordered, Markov). Results confirm: Random > DNA > Indus/English/Tamil/Sanskrit > Fortran > Ordered, consistent with the published findings.

3.2 Decipherment Engine Performance

Test	Accuracy	Details
Synthetic cipher (21 phonemes)	21/21 = 100%	Random substitution, 500 inscriptions, CVC grammar
Ugaritic Baal Cycle (30 signs)	29/30 = 96.7%	Real ancient script, 83 lines, Kandles confidence 1.000

Table 2. Decipherment accuracy.

3.3 Indus Script Hypothesis Test

We tested proto-Dravidian vs Vedic Sanskrit as competing target language hypotheses on a statistically representative synthetic Indus corpus (6,823 signs, 417 unique, matching published Zipf-Mandelbrot distributions from Yadav et al. 2010).

Hypothesis	Score	Word Matches	Kandles
Proto-Dravidian	297.0	28 (28% coverage)	0.985
Vedic Sanskrit	77.0	6 (4.9% coverage)	0.976

Table 3. Hypothesis engine results on synthetic Indus corpus. Proto-Dravidian scores 4× higher.

Note: These results are on a synthetic corpus that reproduces published statistical properties, not the actual M77/ICIT data. Validation on real inscriptions is the critical next step.

4. What We Need: The ICIT Corpus

Our toolkit is validated and ready. The single critical blocker is access to the actual Indus corpus in machine-readable form. The ICIT database (Wells & Fuls) contains 4,537 inscribed objects with 5,509 texts and 19,616 sign occurrences — exactly the data our pipelines are designed to process.

What we would do with ICIT access:

1. Run all 11 analysis pipelines on the complete corpus
2. Validate our synthetic results against real sign distributions
3. Run the hypothesis engine (Dravidian vs Sanskrit) on real data
4. Apply numeral identification to reduce the phonetic search space
5. Run positional analysis to classify all 676 signs by function
6. Detect paradigms (inflectional patterns) across real inscriptions
7. Build co-occurrence networks to identify sign communities
8. Generate comprehensive PDF reports with all findings

We have already built a Fuls-notation parser that handles the +sign-sign-sign+ format used in the published corpus volumes, including metadata extraction (findspot, object type, iconography, reading direction).

5. Decipherment Methodology

Stage 1 — Structural analysis: Block entropy confirms the script is linguistic. Positional analysis classifies signs by grammatical function. Paradigm detection reveals inflectional patterns. Numeral identification reduces the search space.

Stage 2 — Hypothesis testing: Competing target language models (proto-Dravidian from DEDR roots + Old Tamil; Vedic Sanskrit from Rigveda) are tested against the corpus. The hypothesis that produces more word matches and better paradigm regularity is promoted.

Stage 3 — Iterative decipherment: Confident sign mappings are locked. Remaining signs are optimised against the target language model using bigram/trigram hill climbing with Kandles phonetic validation. Each iteration reduces the unknown space.

6. Technical Details

102 automated tests • Python 3.12 • FastAPI backend • SQLite database • CI on Windows/Linux/macOS • PDF report generation • Cross-platform • Proprietary — incorporates patented technologies (US 2024/0248922 A1, US Provisional 63/980,251)

References

- [1] Rao et al. (2009). Science 324:1165.
- [2] Yadav et al. (2010). PLoS ONE 5(3):e9506.
- [3] Mahadevan (1977). The Indus Script. ASI No. 77.
- [4] Fuls (2023). Corpus of Indus Inscriptions.
- [5] Fuls (2023). A Catalog of Indus Signs.
- [6] Parpola (1994). Deciphering the Indus Script.
- [7] Merkur (2024). US 2024/0248922 A1.